RAMDEOBABA UNIVERSITY, NAGPUR | Department of Electronics Engineering

Deep Learning Lab | Capstone Project | Group 19 | Session 2026-27, Semester V

Fresh-versus-Rotten Fruit Classification

Complete Instruction Manual

Baseline CNN vs MobileNetV2 transfer learning
Colab data preparation - DGX B200 training - Gradio application

Document	What it covers
This manual	which file to open, where to run it (Colab / DGX B200 / laptop), exact commands, expected outputs, troubleshooting, final checklist
README.md	short overview and quick start
docs/	METHODOLOGY, DGX_B200_GUIDE, DESIGN_DECISIONS, PROPOSAL_TRACEABILITY, TROUBLESHOOTING, TESTING_STATUS
notebooks/	four executable notebooks (01 Colab, 02-04 DGX or laptop)

WARNING
This project package contains **no trained models and no accuracy, precision, recall or F1 numbers**. They appear only after you run the training on the DGX. The code checks the proposal targets (accuracy at least 90 %, macro-F1 above 0.85) against your real results and prints PASS or FAIL - it never assumes success.

Contents

Contents 2

1. Project at a glance 3

2. Design in brief 4

2.1 Why the data split is special . . . 4

2.2 How the models are judged . . . 4

3. What is in the project folder 5

4. Complete execution order 6

5. Phase A - Google Colab: data preparation 7

5.1 Open the notebook . . . 7

5.2 Run all cells - what each section does . . . 7

5.3 Alternative: run without the notebook . . . 7

6. Phase B - move the data to the DGX 8

7. Phase C - DGX B200: environment and training 9

7.1 Environment (choose ONE route) . . . 9

7.2 Check the GPUs . . . 9

7.3 Train Model 1 - baseline CNN . . . 9

7.4 Train Model 2 - MobileNetV2 . . . 9

7.5 Hardware options . . . 10

7.6 Slurm and long jobs . . . 10

7.7 Optional experiments on the DGX . . . 10

8. Phase D - evaluation, comparison and error analysis 11

8.1 How to read the results . . . 11

9. Phase E - the application 12

9.1 Bring the results back and install . . . 12

9.2 Launch . . . 12

9.3 Running the app on the DGX . . . 12

9.4 Command line and notebook alternatives . . . 12

10. Troubleshooting guide 14

11. Final verification checklist 15

12. What was tested, and what needs the college GPU server 15

Appendix A - Command-line reference (generated from the scripts) 17

Appendix B - Glossary 23

1. Project at a glance

The system takes a photo of a fruit and answers three things: **which of six classes** it belongs to (fresh/rotten apple, banana, orange), **whether the fruit is fresh or rotten**, and **how confident** the model is. Two models are trained on the same data and compared fairly: Model 1 is a CNN built from scratch (the baseline); Model 2 is MobileNetV2 with transfer learning. The proposal targets are at least 90 % accuracy and an F1-score above 0.85 for both models - these are tested against the real results.

Phase	Where	What you do	Main file
A. Data preparation	**Google Colab**	download, explore, validate, preview augmentation, split 70/15/15, verify, make bundle	notebooks/01_colab_data_preparation.ipynb
B. Transfer	your computer	move the bundle to the DGX and check it arrived intact	scp, scripts/verify_data.py
C. Training	**DGX B200**	train both models on all suitable GPUs; optional experiments	scripts/train.py or notebooks/02_dgx_training.ipynb
D. Evaluation	DGX	metrics, confusion matrices, fair comparison, PASS/FAIL, error analysis	scripts/evaluate.py, compare.py, error_analysis.py, notebooks 03-04
E. Application	laptop (or DGX)	upload an image, see class, fresh/rotten status and confidence	app/app.py

Figure 1 - System architecture and where each part runs

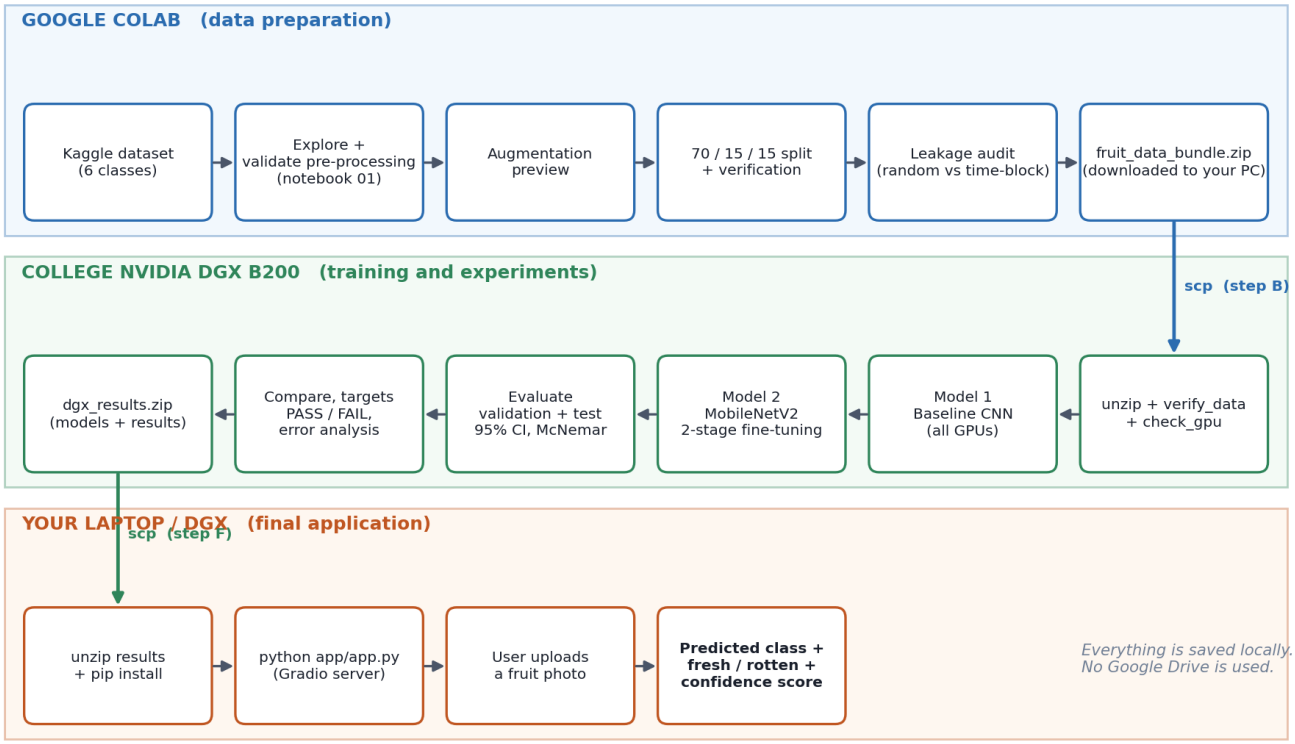


Figure 1 - Architecture of the whole system and where every part runs.

Important assumptions. The GPU specifications and the Colab/DGX workflow you mentioned were not available when this package was created. The package therefore assumes: a DGX B200 node with several NVIDIA B200 (Blackwell) GPUs, shell/SSH access, and an environment that you set up (NGC container or virtual environment). Nothing is hard-coded: GPU count, batch size and precision are command-line options (section 7 and docs/DGX_B200_GUIDE.md). If your site differs (for example Slurm, no internet), use the matching alternative given in each step.

2. Design in brief

Figure 2 - The two model architectures

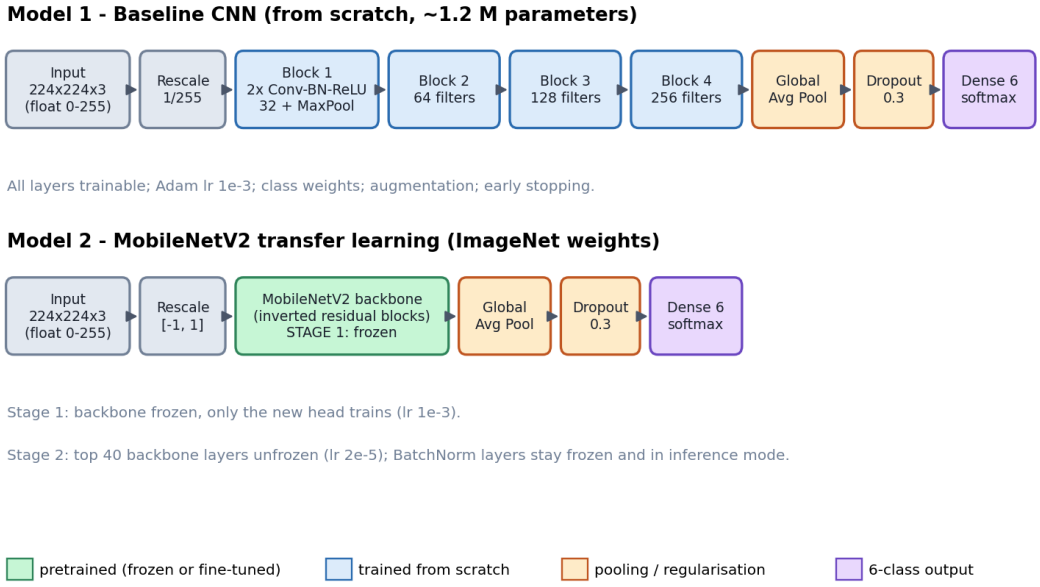


Figure 2 - Model 1 (baseline CNN) and Model 2 (MobileNetV2, two training stages).

2.1 Why the data split is special

The Kaggle data holds only about 1,500 *original* photos; most of the roughly 13,600 files are rotated, flipped, shifted or noisy *copies*, and the originals are screenshots taken seconds apart. If files are split at random, copies and near-duplicates of the same fruit land in both training and test data and the accuracy looks excellent for the wrong reason. Notebook 01 re-measures these facts on your copy of the data. The project therefore splits **source photos in time blocks** with a 60-second gap between train, validation and test (seed 42). The result is close to - not exactly - 70/15/15 of the images, because whole blocks are assigned and the buffer removes some photos.

Figure 3 - Time-block split (schematic illustration, not real data)

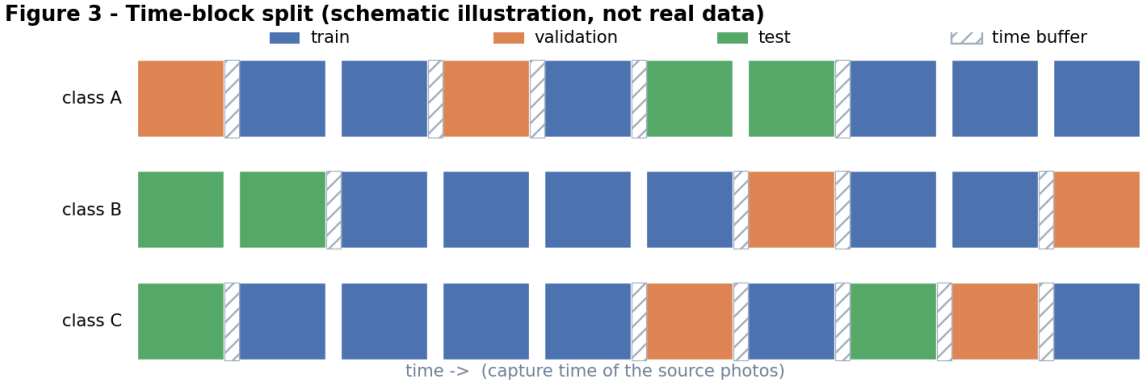


Figure 3 - Time-block split: whole blocks go to one split; a buffer separates neighbours.

2.2 How the models are judged

- **Primary test view:** one original image per source photo (independent samples). **Secondary:** all test images including augmented copies.
- Accuracy, precision, recall, F1 (per class and macro), confusion matrices, 95 % bootstrap intervals, exact McNemar test.
- Fairness: same split, seed, input size, augmentation, class weights, loss and optimiser for both models (a table proves it).
- Size and speed: parameters, file size, latency per image, optional TensorFlow Lite export - the proposal asks which model suits limited hardware.

- Final model for the application: chosen by a rule fixed in advance using the **validation** split only.

3. What is in the project folder

```
fresh-rotten-fruit-capstone/
  README.md  Makefile  requirements*.txt  pyproject.toml  LICENSE
  configs/     default.json  dgx_b200.json  single_gpu_debug.json      <- hardware profiles
  notebooks/   01_colab_data_preparation.ipynb   (Colab)
               02_dgx_training.ipynb             (DGX)
               03_evaluation_comparison.ipynb    (DGX / laptop)
               04_error_analysis_and_app.ipynb   (DGX / laptop)
  scripts/     prepare_data  download_dataset  verify_data  check_gpu  fetch_pretrained_weights
               train  run_experiments  summarize_experiments  evaluate  compare  error_analysis
               export_tflite  predict  package_results  run_dgx_pipeline.sh
  slurm/       train_dgx.sbatch                 <- optional, only if your DGX uses Slurm
  app/         app.py                           <- the web application
  src/fruitfresh/   the Python package used by all scripts and notebooks
  tests/       automated tests (synthetic data)
  docs/        guides + this manual + figures
  data/splits/ split CSVs + split_manifest.json   (created in step A)
  data/raw/    dataset images on the DGX            (created in step B)
  workspace/   trained runs: models, histories, predictions   (created in step C)
  results/     tables, figures, reports, final_model.json     (created in step D)
```

4. Complete execution order

Follow the steps in this order. Each row says **where** to run it and **which file** to open or which command to type. Details and expected outputs follow in sections 5-9.

#	Where	File / command	Result you should see
A1	Colab	Upload *fresh-rotten-fruit-capstone.zip* to /content (Files panel); open *notebooks/01_colab_data_preparation.ipynb*	notebook opens
A2	Colab	Runtime > Run all (CPU is enough)	last cell prints *READY FOR THE DGX*
A3	Colab	browser downloads *fruit_data_bundle.zip*	file on your computer
B1	your PC	`scp fruit_data_bundle.zip user@dgx:path/fresh-rotten-fruit-capstone/`	file copied
B2	DGX	`unzip -o fruit_data_bundle.zip`	data/splits/ (and data/raw/) exist
B3	DGX	`python scripts/verify_data.py --split-dir data/splits --data-root data/raw`	*DATA OK*
C1	DGX	create the environment (section 7.1), then `python scripts/check_gpu.py`	*GPU CHECK PASSED*
C2	DGX (offline only)	`python scripts/fetch_pretrained_weights.py` on a PC with internet; copy assets/*.h5 to the DGX	weights file in assets/
D1	DGX	`python scripts/train.py --model baseline_cnn --config configs/default.json`	*[baseline_cnn] done in ...*
D2	DGX	`python scripts/train.py --model mobilenet_v2 --config configs/default.json`	*[mobilenet_v2] done in ...*
D3	DGX (optional)	`python scripts/run_experiments.py --family mobilenet_v2 --names head_only ft_top20`	extra runs in workspace/runs/
E1	DGX	`python scripts/evaluate.py`	one line per model and test view
E2	DGX	`python scripts/compare.py`	metrics table, PASS/FAIL table, final model
E3	DGX	`python scripts/error_analysis.py --run baseline_cnn` (and mobilenet_v2)	results/error_analysis/...
E4	DGX	`python scripts/package_results.py --out dgx_results.zip`	dgx_results.zip
F1	your PC	`scp user@dgx:path/.../dgx_results.zip .`; `unzip -o dgx_results.zip -d fresh-rotten-fruit-capstone`	workspace/runs and results/ appear
F2	laptop	`pip install -r requirements.txt`; `python app/app.py`	browser: http://127.0.0.1:7860
G	anywhere	go through the final checklist (section 11)	all boxes ticked

TIP
Everything on the DGX can be run in one go with `bash scripts/run_dgx_pipeline.sh` (steps 3-8 above; optional environment variables PROFILE, DATA_ROOT, EXPERIMENTS=1, OUT_ZIP, ALLOW_CPU=1, EXTRA_ARGS). Run long jobs inside `tmux` so a dropped SSH connection does not stop them. Finished runs are detected and never retrained by accident.

5. Phase A - Google Colab: data preparation

RUN ON
Google Colab (CPU runtime is enough). File: **notebooks/01_colab_data_preparation.ipynb**.

5.1 Open the notebook

- Go to colab.research.google.com. In the left **Files** panel upload *fresh-rotten-fruit-capstone.zip* (it must end up as /content/fresh-rotten-fruit-capstone.zip).
- Upload the notebook too (File > Upload notebook) **or** open it from GitHub if you pushed the project there and set REPO_URL in the first cell.
- The first cell (SETUP) unzips the project, installs the few extra packages (Colab already has TensorFlow) and prints the project paths.

5.2 Run all cells - what each section does

Section	What happens	What to look for
1 Download	kagglehub downloads the public dataset	prints *Dataset root: ...* and the folders train, test
2 Exploration	index of every file: class, source photo, original or augmented copy, capture time	files vs source photos: far fewer sources than files; class balance chart; sample images
3 Pre-processing	unreadable-file check, image-size statistics, one tensor batch	*Unreadable images: 0* (or a short list); batch shape (8, 224, 224, 3), range 0-255
4 Augmentation	preview of the training-time augmentation	grid of varied versions of one photo
5 Split	70/15/15 time-block split, verification (raises an error if anything leaks)	image and source counts per split, minimum gap at least 60 s, timeline plot
6 Leakage audit	random split vs time-block split similarity	two summary lines + histogram
7 Bundle	saves data/splits/*.csv + manifest, creates fruit_data_bundle.zip, downloads it	browser download; *READY FOR THE DGX*

EXPECTED OUTPUT
The achieved percentages of the split are printed by section 5 (images per split divided by the total). They should be close to 70 / 15 / 15; a deviation of a few points is normal (whole blocks + time buffer). The numbers in your run are the ones that count.

WARNING
INCLUDE_IMAGES (last section): keep **True** if the DGX has no internet (the zip then contains the images - it can be large). Set **False** to get a tiny zip with only the CSVs; the DGX then downloads the images itself (step 5.3).

5.3 Alternative: run without the notebook

```
python scripts/prepare_data.py --data-root <folder with train/ and test/> --out data/splits \
    --audit \
       --bundle fruit_data_bundle.zip            # add --bundle-no-images for a CSV-only bundle
```

NOT TESTED HERE
Real Google Colab and the real Kaggle download could not be tested in the development environment. The same code was run on a synthetic dataset with identical structure and file naming; see section 12.

6. Phase B - move the data to the DGX

Situation	What to do
DGX has **no internet**	Use the bundle **with images**: `scp fruit_data_bundle.zip user@dgx:path/fresh-rotten-fruit-capstone/`, then on the DGX `unzip -o fruit_data_bundle.zip`. Images appear in data/raw/, CSVs in data/splits/.
DGX **has internet**	Copy only the CSV bundle (INCLUDE_IMAGES = False), unzip it, then `python scripts/download_dataset.py --link data/raw`.
Dataset already on the DGX	`export FRUIT_DATA_ROOT=/path/containing/train-and-test` or pass `--data-root` to every script.

Always verify after the transfer. It checks the SHA-256 hash of every split CSV against the manifest and that every listed image exists:

```
python scripts/verify_data.py --split-dir data/splits --data-root data/raw
# add --decode to open every image (slow)
```

EXPECTED OUTPUT
A small JSON summary (images and sources per split, *missing: 0*) followed by **DATA OK**. Anything else prints *DATA PROBLEM: ...* and exits with an error - fix it before training (section 10).

7. Phase C - DGX B200: environment and training

RUN ON
The college NVIDIA DGX B200, from the project folder, in a terminal (SSH) or JupyterLab. Notebook alternative: **notebooks/02_dgx_training.ipynb**.

7.1 Environment (choose ONE route)

Route	Commands	When
A. NGC TensorFlow container	`docker run --gpus all -it --rm --shm-size=16g -v "$PWD":/workspace -w /workspace <ngc-tensorflow-image> bash` `pip install -r requirements-train.txt`	recommended for Blackwell GPUs; choose a release whose notes mention B200/Blackwell; your site may use Singularity/Enroot instead of Docker
B. virtual environment	`python3 -m venv .venv && source .venv/bin/activate` `pip install tensorflow && pip install -r requirements-train.txt`	only if that TensorFlow build supports the B200 (CUDA 12.8 or newer)
C. conda	same idea as B inside a conda environment	if your site prefers conda

WARNING
The authors could **not** test on a B200. If TensorFlow does not see the GPUs, or reports *no kernel image*, the TensorFlow build is not compatible with Blackwell - use route A or ask the DGX administrators which TensorFlow/CUDA combination is supported.

7.2 Check the GPUs

```
python scripts/check_gpu.py
```

EXPECTED OUTPUT
TensorFlow and CUDA versions, *GPUs visible to TensorFlow: N*, one line per GPU, *GPU:i matmul OK* for each, *MirroredStrategy all-reduce OK* when N is above 1, a bfloat16 line, and finally **GPU CHECK PASSED**. Do not continue until you see it.

7.3 Train Model 1 - baseline CNN

```
python scripts/train.py --model baseline_cnn --config configs/default.json
```

The first line printed is the hardware summary (devices, number of replicas, global batch size, precision). Then one line per epoch. At the end:

```
[baseline_cnn] done in <minutes> min on <N> x GPU, best val_loss <value>
```

Files created in `workspace/runs/baseline_cnn/seed42/`: **model.keras** (final model), **history.csv** (curves), **train_info.json** (settings, hardware, timings, library versions). Running the same command again prints *already trained* and does nothing (use `--force` to retrain).

7.4 Train Model 2 - MobileNetV2

```
python scripts/train.py --model mobilenet_v2 --config configs/default.json
# offline DGX:  add  --weights-file
    assets/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
```

Stage 1 (head only) runs first, then stage 2 (fine-tuning). **train_info.json** records, for each stage, how many layers and parameters were trainable and that *backbone_batchnorm_trainable* is 0. After stage 1 the model is saved as *stage1.keras*; if the job dies later, running the command again resumes from there.

7.5 Hardware options

Option	Meaning	Example
--gpus	all (default), a number, or ids	--gpus 4 or --gpus 0,1
--per-gpu-batch-size	images per GPU; global batch = this x number of GPUs	--per-gpu-batch-size 16
--mixed-precision	off (default), bf16, fp16	--mixed-precision bf16
--lr-scaling	none (default), sqrt, linear: scale learning rate with GPU count	--lr-scaling sqrt
--config	JSON profile; command-line options override it	configs/dgx_b200.json
--set key=value	override a model setting for one run (with --run-name)	--set ft_layers=20 --run-name my_variant

Profiles: *configs/default.json* (all GPUs, 32 images/GPU, full precision) - use it first. *configs/dgx_b200.json* (64 images/GPU, bfloat16) - a performance profile to try after the default works. *configs/single_gpu_debug.json* - one GPU, to tell model problems from multi-GPU problems. The dataset is small: one B200 is plenty; using all GPUs shortens epochs but gives few optimisation steps per epoch (see docs/DGX_B200_GUIDE.md).

7.6 Slurm and long jobs

```
tmux new -s fruit            # then run the commands inside; detach with Ctrl-b d
# Slurm sites: edit the #SBATCH lines, then
sbatch slurm/train_dgx.sbatch
tail -f logs/fruit-train-<jobid>.out
```

Watch the GPUs with `watch -n 2 nvidia-smi`: memory should be in use on every selected GPU.

7.7 Optional experiments on the DGX

```
python scripts/run_experiments.py --family mobilenet_v2 --names head_only ft_top20 ft_top80 \
    ft_lr_1e-5 --config configs/default.json
python scripts/summarize_experiments.py --family mobilenet_v2
```

Each experiment changes **one** thing and is judged on the **validation** split only; the test split is never used to choose. Names are listed in *src/fruitfresh/config.py* (EXPERIMENTS). To compare the best run of each model run both summaries, then `python scripts/compare.py --use-best`.

NOT TESTED HERE
Multi-GPU training was tested only with **two logical CPU replicas** (same MirroredStrategy code path, no NCCL). Real B200 execution, bfloat16 on B200, and timing were not tested.

8. Phase D - evaluation, comparison and error analysis

RUN ON
DGX (or any machine that holds the trained runs). Notebooks: **03_evaluation_comparison.ipynb**, **04_error_analysis_and_app.ipynb**.

```
python scripts/evaluate.py
python scripts/compare.py
python scripts/error_analysis.py --run baseline_cnn
python scripts/error_analysis.py --run mobilenet_v2
```

evaluate.py prints one line per model and test view: accuracy, macro-F1, number of images and source photos, and **targets: PASS/FAIL**. **compare.py** prints the metric table, the PASS/FAIL table against the proposal targets, and the final model for the application.

File (in results/ or the run's eval/ folder)	Content
comparison_report.md	all tables of the comparison, assembled only from saved numbers
metrics_test_original.csv, metrics_test_full.csv	accuracy, macro precision, recall, F1 with 95 % intervals
per_class_f1_*.csv	F1 of each of the six classes for both models
targets_check.csv	accuracy >= 90 % and macro-F1 > 0.85: PASS/FAIL per model and view
efficiency.csv	parameters, file size, latency, training time
training_curves.png, metric_bars_*.png	training/validation curves; metric bars with intervals and target lines
final_model.json	which model the application will use, and why
workspace/runs/<run>/seed42/eval/	reports (precision/recall/F1 per class), confusion matrices (counts and normalised), per-image predictions
results/error_analysis/<run>/	most frequent mistakes, mistakes gallery, Grad-CAM, written error report

8.1 How to read the results

- Compare the two models on the **same** view. The primary view (test originals) is the headline; the full view shows robustness to rotated/noisy copies.
- The test set contains a few hundred independent photos, so differences of one or two points may be noise: look at the 95 % intervals and the McNemar p-value in the report (p above 0.05 = could be chance).
- **If a target is missed, report it honestly** and use the error analysis (top confusions, fresh-vs-rotten vs fruit-type mistakes, Grad-CAM) to explain why.
- **If accuracy is 99 % or higher, be suspicious** rather than proud: re-read the leakage audit and the split verification output.
- Which model suits limited hardware? Use the efficiency table and the optional TensorFlow Lite export: `python scripts/export_tflite.py --run mobilenet_v2`.

9. Phase E - the application

RUN ON
Your laptop (recommended for the demonstration) or the DGX. File: **app/app.py**. Needs *workspace/runs/...* and *results/final_model.json* (from *dgx_results.zip*).

9.1 Bring the results back and install

```
# on your laptop
scp user@dgx-host:path/fresh-rotten-fruit-capstone/dgx_results.zip .
unzip -o dgx_results.zip -d fresh-rotten-fruit-capstone
cd fresh-rotten-fruit-capstone
pip install -r requirements.txt
```

Use the same major TensorFlow/Keras version as on the DGX (Keras 3) so the *.keras* file loads.

9.2 Launch

```
python app/app.py                                  # final model chosen by compare.py
python app/app.py --model workspace/runs/mobilenet_v2/seed42/model.keras
python app/app.py --model results/export/mobilenet_v2.tflite   # lightweight TFLite model
python app/app.py --host 0.0.0.0 --share
# reachable from other machines / temporary public link
```

Open **http://127.0.0.1:7860** in the browser. Upload a fruit photo and press **Classify**. The page shows the fruit, **FRESH or ROTTEN**, the predicted class, the confidence of that class, the fresh/rotten confidence, a bar chart of all six probabilities, and a warning when the confidence is below 60 %. Tick the Grad-CAM box (Keras models) to see where the model looked.

Figure 4 - Layout of the application (schematic; the values shown are placeholders)

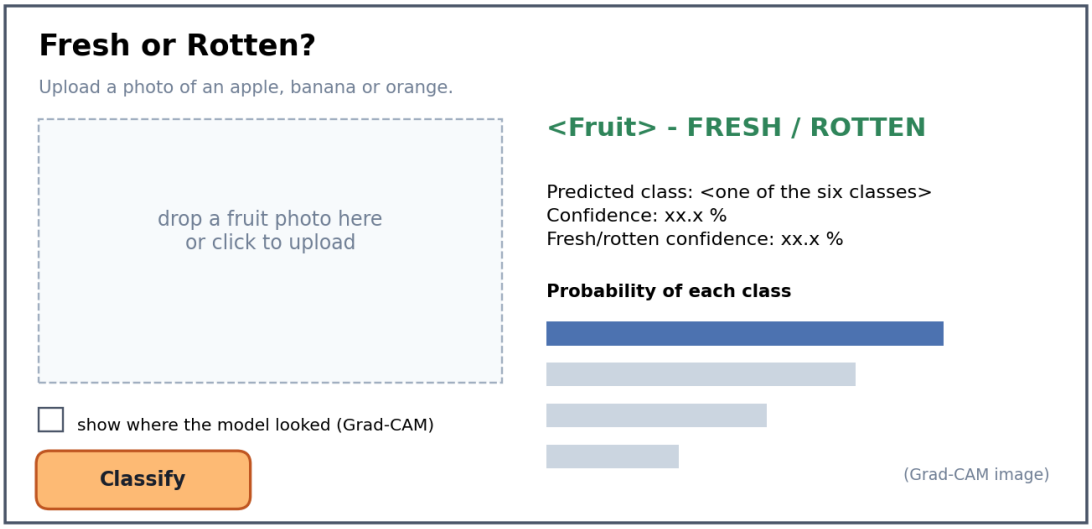


Figure 4 - Layout of the application (schematic; values are placeholders).

9.3 Running the app on the DGX

```
python app/app.py --host 127.0.0.1 --port 7860        # on the DGX
ssh -L 7860:localhost:7860 user@dgx-host                    # on your laptop, second terminal
# then open http://127.0.0.1:7860 on your laptop
```

9.4 Command line and notebook alternatives

```
python scripts/predict.py --model auto --image apple.jpg banana.jpg
# prints class | FRESH/ROTTEN | confidence
python scripts/predict.py --model auto --image apple.jpg --json  # machine-readable
```

Notebook 04 contains the same pipeline plus the real-world photo test: put your own phone photos in `real_world/<class_name>/*.jpg` (class names: freshapples, freshbanana, freshoranges, rottenapples, rottenbanana, rottenoranges). These photos must never be used for training.

TIP
Before the demonstration: run the app once with a fresh photo and a rotten photo of each fruit, and keep a backup of *dgx_results.zip*.

10. Troubleshooting guide

Symptom	Likely cause	Fix
Notebook: *Project folder not found*	zip not uploaded to /content	upload fresh-rotten-fruit-capstone.zip in the Colab Files panel (or set REPO_URL) and re-run the SETUP cell
Kaggle download fails	no internet / rate limit	retry; or download the dataset in a browser, unzip, set FRUIT_DATA_ROOT
ModuleNotFoundError: fruitfresh	wrong folder or SETUP cell skipped	run commands from the project root; run the SETUP cell first
verify_data: *images missing*	bundle not unpacked inside the project, wrong --data-root	unzip in the project folder; check data/raw/train/<class>/ exists
test.csv changed since the split was created	CSV edited or re-saved	re-copy CSVs + split_manifest.json from the bundle; never open them in a spreadsheet
check_gpu: no GPU visible	wrong environment, driver/CUDA mismatch	nvidia-smi; use a Blackwell-capable TensorFlow build or NGC container (section 7.1)
no kernel image is available	TensorFlow build lacks B200 kernels	use a TensorFlow/NGC release that supports Blackwell (CUDA 12.8+)
Out of memory	batch too large / GPU shared	--per-gpu-batch-size 16; check nvidia-smi for other users
Hang or NCCL error at start of multi-GPU training	GPU communication	--config configs/single_gpu_debug.json to isolate; NCCL_DEBUG=INFO; ask the administrators
Training slow, GPUs idle	data on slow disk / few CPU cores	copy data to local SSD; allocate more CPU cores; epoch 1 is slower (images get cached)
MobileNetV2: weights download fails	DGX offline	fetch_pretrained_weights.py elsewhere; copy assets/*.h5; use --weights-file
Accuracy around 17 % and flat	wrong labels/paths, bad settings	check train_info.json and history curves; retrain with --force
Test accuracy 99 % or more	possible leakage / easy test set	read the leakage audit; do not present as success without analysis
One class has very low recall	similar classes (e.g. rotten vs fresh orange), few test blocks	error_analysis.py: top confusions and Grad-CAM; consider deeper fine-tuning
compare.py --use-best fails	results/best_*.json missing	run summarize_experiments.py for both families first
App: model/final_model.json not found	compare.py not run or results not copied	run compare.py or pass --model <path>
App: port in use	another Gradio instance	python app/app.py --port 7861
.keras file will not load	different Keras version	use TensorFlow 2.16 or newer (Keras 3) on both machines
Colab runtime reset, files gone	Colab disk is temporary	re-run SETUP; keep the downloaded bundle on your computer; for runs use backup_workspace / restore_workspace

11. Final verification checklist

	Check	How to verify
☐	Dataset downloaded and explored	notebook 01 sections 1-3 printed without errors; unreadable images = 0
☐	Split created and verified	verify_split passed; minimum gap at least 60 s; counts close to 70/15/15
☐	Bundle on the DGX is intact	verify_data.py prints DATA OK
☐	All GPUs usable	check_gpu.py prints GPU CHECK PASSED
☐	Model 1 trained	workspace/runs/baseline_cnn/seed42/model.keras, history.csv, train_info.json exist
☐	Model 2 trained in two stages	stage1.keras and model.keras exist; train_info.json: finetune stage, backbone_batchnorm_trainable = 0
☐	Training curves look sensible	results/training_curves.png: validation loss falls; no huge gap
☐	Both models evaluated on the same data	eval/metrics.json in both runs; fairness table identical except the model
☐	Accuracy, precision, recall, F1 reported	results/metrics_*.csv, per_class_f1_*.csv, report_*.txt
☐	Confusion matrices saved	eval/confusion_*.png (counts and normalised)
☐	Proposal targets checked	results/targets_check.csv shows PASS or FAIL honestly for each model
☐	Comparison report generated	results/comparison_report.md incl. McNemar test and efficiency table
☐	Error analysis done	results/error_analysis// has the report, gallery and Grad-CAM
☐	Final model chosen by the fixed rule	results/final_model.json
☐	Application runs and gives correct output format	upload a photo: class, FRESH/ROTTEN, confidence shown
☐	Real-world photos tested (recommended)	notebook 04, section 4
☐	Results copied back and backed up	dgx_results.zip on your laptop
☐	Automated tests pass	python -m pytest tests -q (about 10 minutes, CPU)
☐	No result in the report is typed by hand	every number comes from results/ or train_info.json

12. What was tested, and what needs the college GPU server

Tested in the development environment (CPU only, a **synthetic** dataset with the same folder layout and file naming as the Kaggle data):

- data indexing, file-name parsing, exact-duplicate removal, time-block / random / cross-validation splits and their leakage checks
- split CSV hashing, data bundle creation and verification
- both model architectures, the freezing logic (BatchNorm stays frozen), training of both models end to end, resume after interruption
- multi-GPU training code with **two logical CPU replicas** (MirroredStrategy)
- evaluation metrics, bootstrap intervals, McNemar test, comparison report, error analysis, Grad-CAM
- TensorFlow Lite export and agreement with the Keras model
- inference pipeline, command-line prediction, and the **real Gradio server** called through its API (with and without Grad-CAM)
- the one-command DGX shell pipeline (with CPU allowed) and the results package layout

- all four notebooks executed top to bottom in a reduced mode (1 epoch, 64 px images, no ImageNet weights)
- every command and option written in the README/docs exists in the scripts (automated documentation test)

```
Last full run (2026-10-24, CPU only, synthetic stand-in dataset, Python 3.12.3, TensorFlow
    2.21.0):
  automated tests : 52 passed, 0 failed, 0 skipped (of 52), 494 s
  notebooks       : 4 of 4 executed top to bottom (reduced mode)
```

Needs your environment (not tested here):

Item	Why it could not be tested	How you will know it works
Real Kaggle download and the real dataset statistics	no access to Kaggle from the development environment	notebook 01 outputs (counts, audit)
Running on Google Colab	no Colab access	notebook 01 runs to *READY FOR THE DGX*
DGX B200 execution, NCCL multi-GPU, Blackwell compatibility, bfloat16	no GPU server	check_gpu.py passes; training prints the hardware line and finishes
ImageNet weight download	blocked in the test environment	MobileNetV2 stage 1 starts (or use the offline weights file)
All accuracy / F1 numbers and the PASS/FAIL outcome	they exist only after training on real data	scripts/compare.py output
Training time and GPU speed-up	not measured	train_info.json: training_seconds

Appendix A - Command-line reference (generated from the scripts)

The text below is the real `--help` output of each script, captured when this manual was built.

scripts/check_gpu.py

```
usage: check_gpu.py [-h] [--allow-cpu]

Check the GPUs before training (run this FIRST on the DGX B200).

    python scripts/check_gpu.py            # fails (exit 1) if TensorFlow sees no GPU
    python scripts/check_gpu.py --allow-cpu

options:
  -h, --help   show this help message and exit
  --allow-cpu
```

scripts/compare.py

```
usage: compare.py [-h] [--baseline-run BASELINE_RUN] [--mobilenet-run MOBILENET_RUN]
    [--use-best]
                  [--n-boot N_BOOT] [--split-dir SPLIT_DIR] [--data-root DATA_ROOT]
                  [--work-dir WORK_DIR] [--results-dir RESULTS_DIR] [--seed SEED]

Fair comparison of Model 1 (CNN) and Model 2 (MobileNetV2) + final-model choice.

    python scripts/compare.py                                    # baseline_cnn vs mobilenet_v2
python scripts/compare.py --use-best  # use the best experiment run of each family
Writes results/: comparison_report.md, tables (CSV), figures (PNG), final_model.json

options:
  -h, --help            show this help message and exit
  --baseline-run BASELINE_RUN
  --mobilenet-run MOBILENET_RUN
  --use-best            use results/best_<family>.json written by summarize_experiments.py
  --n-boot N_BOOT
  --split-dir SPLIT_DIR
                        folder with train/validation/test.csv
  --data-root DATA_ROOT
folder containing the Kaggle train/ and test/ (default: $FRUIT_DATA_ROOT,
                        data/raw, or Kaggle download)
  --work-dir WORK_DIR   where runs are stored
  --results-dir RESULTS_DIR
                        where tables/figures are written
  --seed SEED
```

scripts/download_dataset.py

```
usage: download_dataset.py [-h] [--link LINK] [--copy]

Download the public Kaggle dataset (needs internet) and optionally link it into data/raw.

    python scripts/download_dataset.py --link data/raw

options:
  -h, --help   show this help message and exit
  --link LINK  create this folder as a symlink to the downloaded dataset
  --copy       copy instead of symlink
```

scripts/error_analysis.py

```
usage: error_analysis.py [-h] --run RUN [--view {test_original,test_full,validation}] [--n N]
                         [--split-dir SPLIT_DIR] [--data-root DATA_ROOT] [--work-dir WORK_DIR]
                         [--results-dir RESULTS_DIR] [--seed SEED]

Error analysis of one evaluated run: top confusions, mistakes gallery, Grad-CAM, written report.

    python scripts/error_analysis.py --run mobilenet_v2
Writes results/error_analysis/<run>/

options:
  -h, --help            show this help message and exit
  --run RUN
  --view {test_original,test_full,validation}
  --n N                 images in the galleries
  --split-dir SPLIT_DIR
                        folder with train/validation/test.csv
  --data-root DATA_ROOT
folder containing the Kaggle train/ and test/ (default: $FRUIT_DATA_ROOT,
                        data/raw, or Kaggle download)
  --work-dir WORK_DIR   where runs are stored
  --results-dir RESULTS_DIR
                        where tables/figures are written
  --seed SEED
```

scripts/evaluate.py

```
usage: evaluate.py [-h] [--runs RUNS [RUNS ...]] [--n-boot N_BOOT] [--force]
                   [--split-dir SPLIT_DIR] [--data-root DATA_ROOT] [--work-dir WORK_DIR]
                   [--results-dir RESULTS_DIR] [--seed SEED]

Evaluate trained runs on validation + test (accuracy, precision, recall, F1, confusion
    matrices, 95% CIs).

    python scripts/evaluate.py --runs baseline_cnn mobilenet_v2
Writes workspace/runs/<run>/seed<seed>/eval/ (metrics.json, reports, predictions, confusion
    matrices).

options:
  -h, --help            show this help message and exit
  --runs RUNS [RUNS ...]
  --n-boot N_BOOT
  --force
  --split-dir SPLIT_DIR
                        folder with train/validation/test.csv
  --data-root DATA_ROOT
folder containing the Kaggle train/ and test/ (default: $FRUIT_DATA_ROOT,
                        data/raw, or Kaggle download)
  --work-dir WORK_DIR   where runs are stored
  --results-dir RESULTS_DIR
                        where tables/figures are written
  --seed SEED
```

scripts/export_tflite.py

```
usage: export_tflite.py [-h] [--run RUN] [--quantization {none,dynamic,float16}]
                        [--n-check N_CHECK] [--split-dir SPLIT_DIR] [--data-root DATA_ROOT]
                        [--work-dir WORK_DIR] [--results-dir RESULTS_DIR] [--seed SEED]

Export a trained run to TensorFlow Lite and check it matches the Keras model.

    python scripts/export_tflite.py --run mobilenet_v2 --quantization dynamic

options:
  -h, --help            show this help message and exit
  --run RUN             run name, 'auto' (= final model) or a .keras path
  --quantization {none,dynamic,float16}
  --n-check N_CHECK
  --split-dir SPLIT_DIR
                        folder with train/validation/test.csv
  --data-root DATA_ROOT
folder containing the Kaggle train/ and test/ (default: $FRUIT_DATA_ROOT,
                        data/raw, or Kaggle download)
  --work-dir WORK_DIR   where runs are stored
  --results-dir RESULTS_DIR
                        where tables/figures are written
  --seed SEED
```

scripts/fetch_pretrained_weights.py

```
usage: fetch_pretrained_weights.py [-h]

Download the MobileNetV2 ImageNet weights once and copy them to assets/ (for offline machines).

Run on a machine WITH internet (Colab / PC), then copy assets/*.h5 to the DGX project folder
    and train with
    python scripts/train.py --model mobilenet_v2 --weights-file assets/<file>.h5

options:
  -h, --help  show this help message and exit
```

scripts/package_results.py

```
usage: package_results.py [-h] [--out OUT] [--all-runs] [--work-dir WORK_DIR]
                          [--results-dir RESULTS_DIR]

Pack everything produced on the DGX into ONE zip to copy back to your laptop.

    python scripts/package_results.py --out dgx_results.zip            # final models + results
python scripts/package_results.py --out dgx_results.zip --all-runs # include every experiment
    model
On the laptop:  unzip dgx_results.zip -d <project folder>

options:
  -h, --help            show this help message and exit
  --out OUT
  --all-runs            include models of every experiment run
  --work-dir WORK_DIR
  --results-dir RESULTS_DIR
```

scripts/predict.py

```
usage: predict.py [-h] [--model MODEL] --image IMAGE [IMAGE ...] [--json] [--split-dir
    SPLIT_DIR]
                  [--data-root DATA_ROOT] [--work-dir WORK_DIR] [--results-dir RESULTS_DIR]
                  [--seed SEED]

Command-line prediction (same pipeline as the web app).

    python scripts/predict.py --model auto --image apple.jpg banana.jpg
python scripts/predict.py --model workspace/runs/mobilenet_v2/seed42/model.keras --image x.jpg
    --json

options:
  -h, --help            show this help message and exit
  --model MODEL         'auto', a run name, or a .keras/.tflite path
  --image IMAGE [IMAGE ...]
  --json
  --split-dir SPLIT_DIR
                        folder with train/validation/test.csv
  --data-root DATA_ROOT
folder containing the Kaggle train/ and test/ (default: $FRUIT_DATA_ROOT,
                        data/raw, or Kaggle download)
  --work-dir WORK_DIR   where runs are stored
  --results-dir RESULTS_DIR
                        where tables/figures are written
  --seed SEED
```

scripts/prepare_data.py

```
usage: prepare_data.py [-h] [--data-root DATA_ROOT] [--out OUT] [--seed SEED]
                       [--method {time_block,random_group}] [--check-images] [--audit]
                       [--bundle BUNDLE] [--bundle-no-images]

Step 1 (Colab or any machine): index the dataset, create the 70/15/15 split, verify, save.

    python scripts/prepare_data.py --data-root /path/to/dataset --out data/splits
python scripts/prepare_data.py --audit --bundle fruit_data_bundle.zip
# + leakage audit + transfer

options:
  -h, --help            show this help message and exit
  --data-root DATA_ROOT
  --out OUT
  --seed SEED
  --method {time_block,random_group}
time_block = official leakage-safe split; random_group = naive comparison
                        split
  --check-images        try to open every image (slow, finds corrupt files)
  --audit               print the near-duplicate leakage audit
  --bundle BUNDLE       also write a zip for transfer to the DGX
  --bundle-no-images    bundle only the split CSVs (DGX downloads the images itself)
```

scripts/run_experiments.py

```
usage: run_experiments.py [-h] --family {baseline_cnn,mobilenet_v2} [--names NAMES [NAMES ...]]
                          [--split-dir SPLIT_DIR] [--data-root DATA_ROOT] [--work-dir WORK_DIR]
                          [--results-dir RESULTS_DIR] [--seed SEED] [--config CONFIG]
                          [--gpus GPUS] [--per-gpu-batch-size PER_GPU_BATCH_SIZE]
                          [--mixed-precision {off,bf16,fp16}] [--lr-scaling {none,sqrt,linear}]
                          [--weights-file WEIGHTS_FILE] [--no-pretrained] [--smoke] [--force]

Experiments on the DGX: one-change variants of a model, judged on VALIDATION only.

    python scripts/run_experiments.py --family mobilenet_v2 --names head_only ft_top20 ft_top80
python scripts/run_experiments.py --family baseline_cnn --names all --config
    configs/dgx_b200.json
Then:  python scripts/summarize_experiments.py --family mobilenet_v2

options:
  -h, --help            show this help message and exit
  --family {baseline_cnn,mobilenet_v2}
  --names NAMES [NAMES ...]
                        experiment names or 'all'
  --split-dir SPLIT_DIR
                        folder with train/validation/test.csv
  --data-root DATA_ROOT
folder containing the Kaggle train/ and test/ (default: $FRUIT_DATA_ROOT,
                        data/raw, or Kaggle download)
  --work-dir WORK_DIR   where runs are stored
  --results-dir RESULTS_DIR
                        where tables/figures are written
  --seed SEED
  --config CONFIG       JSON profile, e.g. configs/dgx_b200.json
  --gpus GPUS           "all" (default), a number, or ids like "0,1,2"
  --per-gpu-batch-size PER_GPU_BATCH_SIZE
  --mixed-precision {off,bf16,fp16}
  --lr-scaling {none,sqrt,linear}
  --weights-file WEIGHTS_FILE
                        local MobileNetV2 ImageNet weights (offline machines)
  --no-pretrained       train MobileNetV2 from random weights (tests only)
  --smoke               tiny/fast run for testing the pipeline
  --force               retrain even if a finished run exists
```

scripts/summarize_experiments.py

```
usage: summarize_experiments.py [-h] --family {baseline_cnn,mobilenet_v2} [--split-dir
    SPLIT_DIR]
                                [--data-root DATA_ROOT] [--work-dir WORK_DIR]
                                [--results-dir RESULTS_DIR] [--seed SEED]

Validation table of the default run + experiments of a model; picks the best run (validation
    only).

    python scripts/summarize_experiments.py --family mobilenet_v2
Writes results/experiments_<family>.csv and results/best_<family>.json (used by compare.py
    --use-best).

options:
  -h, --help            show this help message and exit
  --family {baseline_cnn,mobilenet_v2}
  --split-dir SPLIT_DIR
                        folder with train/validation/test.csv
  --data-root DATA_ROOT
folder containing the Kaggle train/ and test/ (default: $FRUIT_DATA_ROOT,
                        data/raw, or Kaggle download)
  --work-dir WORK_DIR   where runs are stored
  --results-dir RESULTS_DIR
                        where tables/figures are written
  --seed SEED
```

scripts/train.py

```
usage: train.py [-h] --model {baseline_cnn,mobilenet_v2} [--run-name RUN_NAME]
                [--set [KEY=VALUE ...]] [--split-dir SPLIT_DIR] [--data-root DATA_ROOT]
[--work-dir WORK_DIR] [--results-dir RESULTS_DIR] [--seed SEED] [--config CONFIG]
                [--gpus GPUS] [--per-gpu-batch-size PER_GPU_BATCH_SIZE]
                [--mixed-precision {off,bf16,fp16}] [--lr-scaling {none,sqrt,linear}]
                [--weights-file WEIGHTS_FILE] [--no-pretrained] [--smoke] [--force]

Train Model 1 (baseline CNN) or Model 2 (MobileNetV2) - on one GPU or all GPUs of the DGX.

    python scripts/train.py --model baseline_cnn --split-dir data/splits --data-root data/raw
    python scripts/train.py --model mobilenet_v2 --config configs/dgx_b200.json
python scripts/train.py --model mobilenet_v2 --gpus 2 --set ft_layers=20 --run-name my_variant

options:
  -h, --help            show this help message and exit
  --model {baseline_cnn,mobilenet_v2}
  --run-name RUN_NAME   output folder name (default: the model name)
  --set [KEY=VALUE ...]
                        override model settings, e.g. ft_layers=20 augment=false epochs=5
  --split-dir SPLIT_DIR
                        folder with train/validation/test.csv
  --data-root DATA_ROOT
folder containing the Kaggle train/ and test/ (default: $FRUIT_DATA_ROOT,
                        data/raw, or Kaggle download)
  --work-dir WORK_DIR   where runs are stored
  --results-dir RESULTS_DIR
                        where tables/figures are written
  --seed SEED
  --config CONFIG       JSON profile, e.g. configs/dgx_b200.json
  --gpus GPUS           "all" (default), a number, or ids like "0,1,2"
  --per-gpu-batch-size PER_GPU_BATCH_SIZE
  --mixed-precision {off,bf16,fp16}
  --lr-scaling {none,sqrt,linear}
  --weights-file WEIGHTS_FILE
                        local MobileNetV2 ImageNet weights (offline machines)
  --no-pretrained       train MobileNetV2 from random weights (tests only)
  --smoke               tiny/fast run for testing the pipeline
  --force               retrain even if a finished run exists
```

scripts/verify_data.py

```
usage: verify_data.py [-h] [--split-dir SPLIT_DIR] [--data-root DATA_ROOT] [--work-dir WORK_DIR]
                      [--results-dir RESULTS_DIR] [--seed SEED] [--decode]

Check that the split CSVs and the images are all present and unchanged (run this on the DGX
    after transfer).

    python scripts/verify_data.py --split-dir data/splits --data-root data/raw

options:
  -h, --help            show this help message and exit
  --split-dir SPLIT_DIR
                        folder with train/validation/test.csv
  --data-root DATA_ROOT
folder containing the Kaggle train/ and test/ (default: $FRUIT_DATA_ROOT,
                        data/raw, or Kaggle download)
  --work-dir WORK_DIR   where runs are stored
  --results-dir RESULTS_DIR
                        where tables/figures are written
  --seed SEED
  --decode              also open every image (slow)
```

app/app.py

```
usage: app.py [-h] [--model MODEL] [--host HOST] [--port PORT] [--share]

Final application: upload a fruit photo -> predicted class, fresh/rotten status and confidence.

python app/app.py  # uses the final model chosen by scripts/compare.py
    python app/app.py --model workspace/runs/mobilenet_v2/seed42/model.keras
    python app/app.py --model results/export/mobilenet_v2.tflite --port 7860
python app/app.py --host 0.0.0.0 --share  # reachable from other machines / public link

Works with a Keras (.keras) or TensorFlow Lite (.tflite) model. Grad-CAM ("where did the model
    look?")
is available for .keras models.

options:
  -h, --help     show this help message and exit
  --model MODEL  'auto', a run name, or a .keras/.tflite path
  --host HOST
  --port PORT
  --share        create a temporary public link
```

Appendix B - Glossary

Term	Meaning
Source photo	one original photo; its augmented copies share its identity and always stay in the same split
Time-block split	splitting by capture time in contiguous blocks, with a time buffer between splits
Transfer learning	starting from a network pre-trained on ImageNet and adapting it to our six classes
Fine-tuning	training some of the pre-trained layers with a very small learning rate
MirroredStrategy	TensorFlow's way to train on several GPUs at once with identical copies of the model
Global batch size	images per training step across all GPUs = per-GPU batch x number of GPUs
Macro-F1	average of the F1-scores of the six classes (each class counts equally)
Confidence interval (95 %)	range in which the true value probably lies, computed by resampling source photos
McNemar test	tests whether two models differ on the same images, looking only at images where one is right and the other wrong
Grad-CAM	heat-map showing which image regions influenced the prediction
TFLite	compact model format for phones and small devices